Mastering Asynchronous Streams in Kotlin

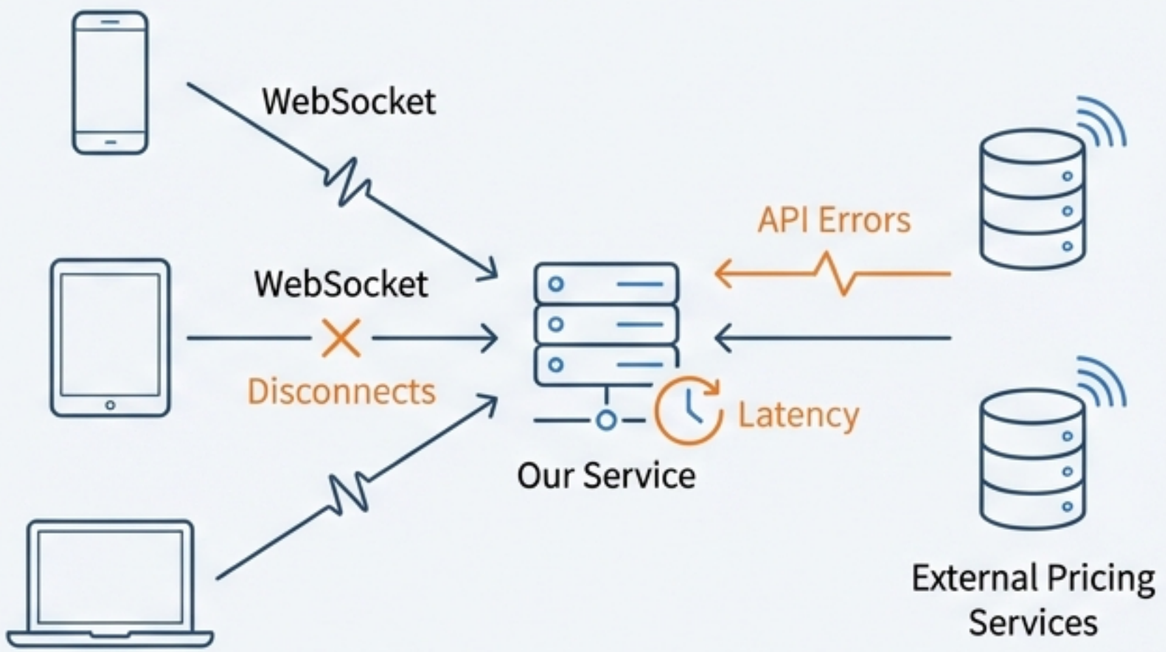
A Strategic Guide to Channels and Flows

This presentation deconstructs advanced coroutine patterns to help you build resilient, high-performance systems.

The Challenge: Building a Real-Time Stock Pricing Service

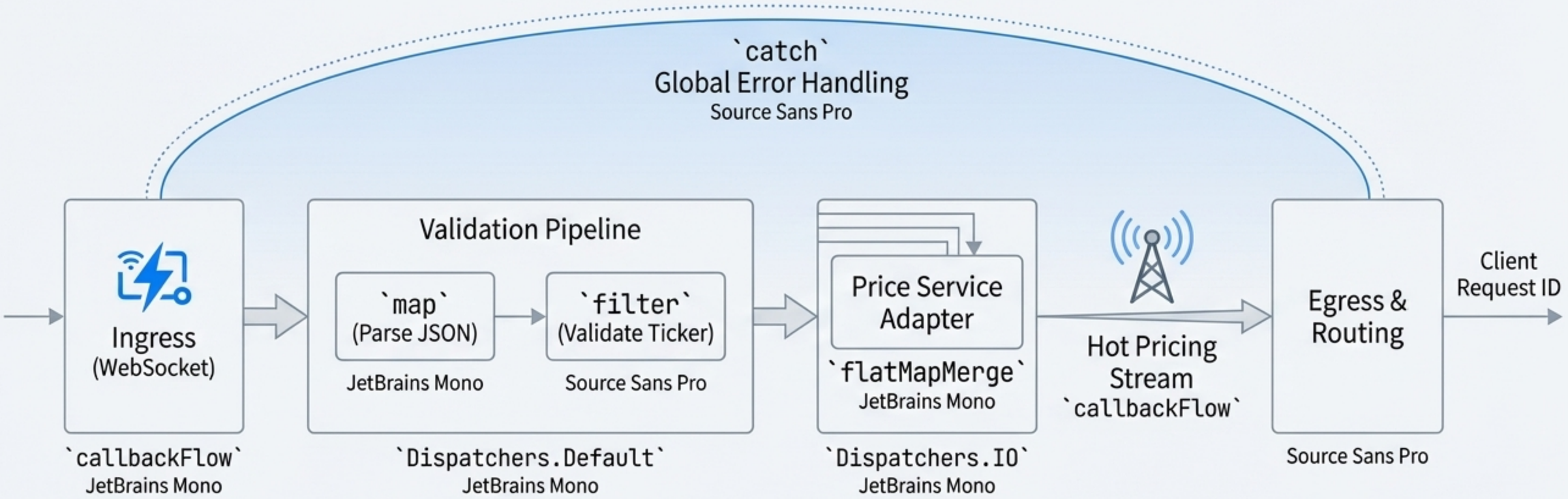
We need to build a service that handles multiple, concurrent client connections via WebSockets. Each client can request real-time price updates for a custom list of stock tickers.
The system must be:

- **Reactive**: Push price updates to clients the moment they are available from an external pricing service (e.g., Reuters, Bloomberg).
- **Concurrent**: Handle hundreds of simultaneous client requests without blocking.
- **Resilient**: Gracefully handle failures from the external pricing service without crashing the entire system.
- **Efficient**: Avoid redundant subscriptions to the pricing service if multiple clients request the same ticker.



This is a non-trivial problem that pushes the limits of traditional asynchronous programming models.

The Blueprint: A Modular, Reactive Assembly Line



Think of this as a modular assembly line. Raw materials (requests) arrive, are inspected (validated), processed by specialists (price adapter), and merged onto a master conveyor belt for delivery.

We will deconstruct this architecture piece by piece, revealing how Kotlin Flows create a powerful, declarative solution.

Step 1: Bridging the Outside World with `callbackFlow`

Problem Statement
How do we adapt a “hot,” callback-based source like a WebSocket listener into a structured Flow?

Solution
The `callbackFlow` builder is designed for this. It converts callback-based APIs into a flow.

Core Concepts

- Internally, `callbackFlow` uses a Channel to buffer incoming events.
- `trySend(event)`: Pushes a new event into the flow from the callback. It's non-suspending and safe to call from any context.
- `awaitClose { }`: A mandatory cleanup block. This is where you unregister listeners to prevent resource leaks when the flow is cancelled or the client disconnects.

```
fun listenForRequests(socket: WebSocket): Flow<RequestEvent> =
callbackFlow {
    val listener = WebSocketListener { json ->
        val request = parse(json) // Parse and create event
        trySend(request)
        trySend(request)
    }
    socket.register(listener)

    // Crucial cleanup logic
    awaitClose { socket.unregister(listener) }
}
```

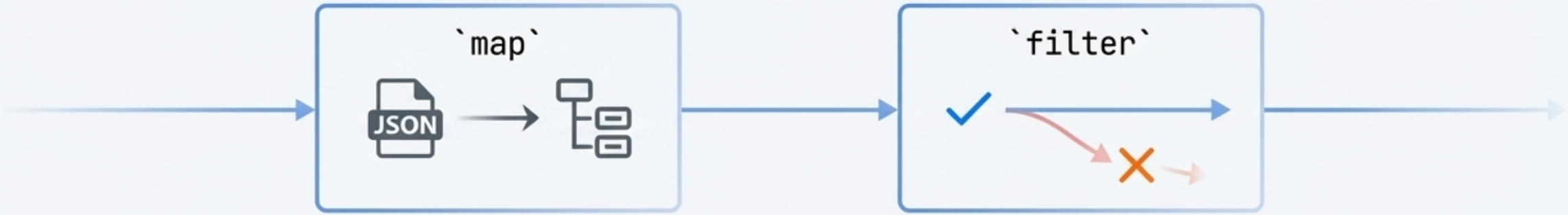
Push event into the Flow

Ensures no resource leaks

`callbackFlow` is the essential bridge for integrating external, asynchronous event sources into a reactive pipeline.

Step 2: Shaping the Stream with Intermediate Operators

Once we have a `Flow`, we can apply a chain of intermediate operators to create a processing pipeline. These operations are lazy and only execute when a value is emitted.



Managing Concurrency

- JSON parsing and validation are CPU-intensive tasks. We must not block the network threads.
- The `flowOn(Dispatchers.Default)` operator changes the execution context for all *upstream* operations.

```
listenForRequests(socket)
    .map { json -> parseRequest(json) }
    .filter { request -> isValid(request.ticker) }
    .flowOn(Dispatchers.Default)
    .collect { ... } // Downstream runs in the collector's context
```

CPU-bound work

Run the above on a background thread pool

Flow operators provide a declarative, non-blocking way to transform data, while `flowOn` gives you precise control over execution context.

A Fundamental Distinction: Cold vs. Hot Data Streams

Cold Flow: A Private Playback

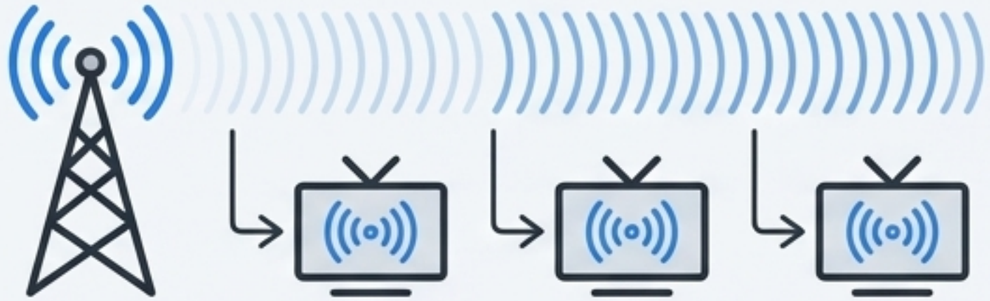
Like a YouTube video. The producer code runs *only when you hit play* (`collect`).

Each collector gets its own, new execution of the producer. It's lazy and unicast.

```
flow { emit(data) }
```

Hot Flow: A Live Broadcast

Like a live television broadcast. The producer is always "on-air," emitting values regardless of whether anyone is watching.



Multiple collectors can tune in and will receive the same stream of values from the moment they start listening. It's proactive and multicast.

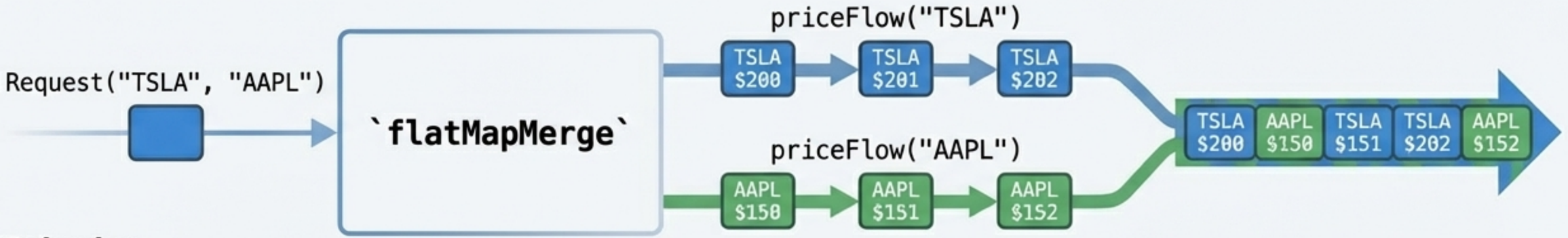
```
SharedFlow, StateFlow.
```

Understanding this distinction is critical to choosing the right tool for managing data distribution and state.

Step 3: Unleashing Concurrency with `flatMapMerge`

A single request for `[TSLA, AAPL, GOOG]` needs to be transformed into three *separate*, *long-running*, *concurrent* price streams.

The `flatMapMerge` operator is designed for this. For each item from the upstream flow, it creates a new flow and merges their emissions into a single downstream flow concurrently.



Behavior

- It does not wait for one inner flow to complete before starting the next.
- Ideal for real-time data where multiple sources need to be processed in parallel.

```
// requestFlow: emits requests for multiple tickers
requestFlow.flatMapMerge { request ->
    // For each request, create a new Flow that streams prices
    getPriceStreamFor(request.tickers) // Returns another Flow<PriceUpdate>
}
```

`flatMapMerge` **is the key to transforming single events into multiple concurrent, real-time data streams.**

Step 4: Building Resilience with the `catch` Operator

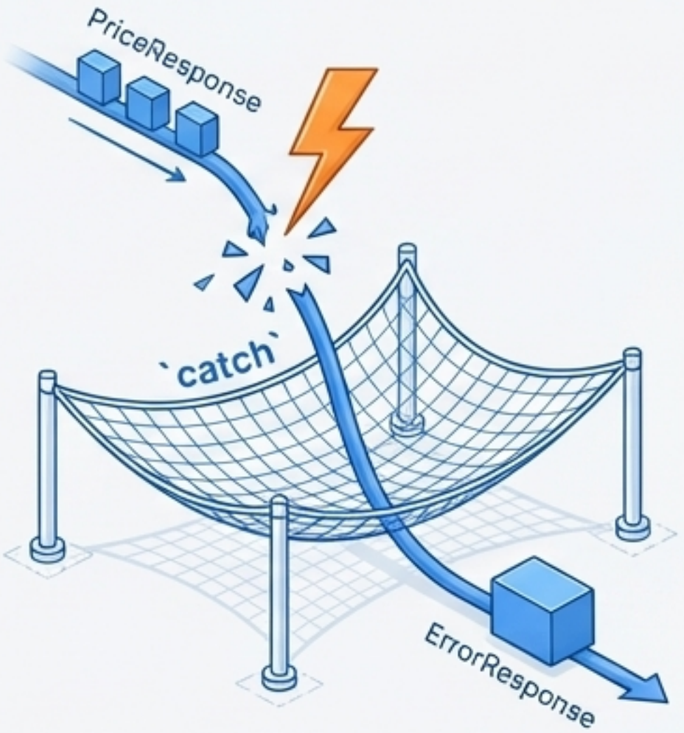
The Danger

An exception in the pricing service (e.g., network error, invalid ticker) could crash the entire flow, terminating the client's connection.

The Solution

The `catch` operator intercepts exceptions from the upstream flow. It provides a block where you can handle the error, log it, and even `emit` a final, alternative value.

```
getPriceStreamFor(request.tickers)
    .map { price -> PriceResponse(price) }
    .catch { exception ->
        // The price stream failed! Don't crash.
        logError(exception)
        // Emit a special error event to the client.
        emit(ErrorResponse("Subscription failed for ${request.ticker}"))
    }
```

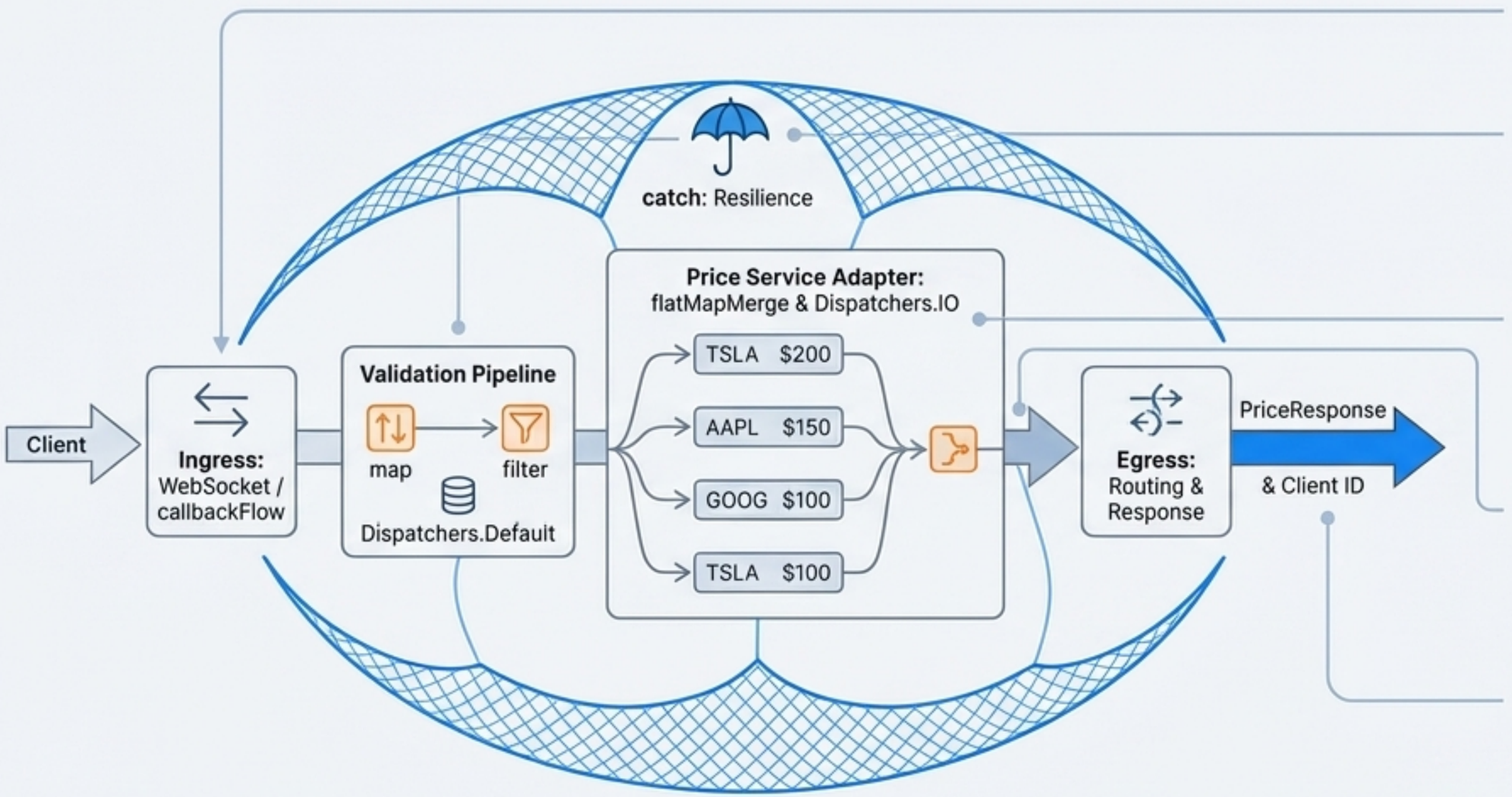


Behavior

- `catch` is a terminal operator for the upstream flow; after an exception, the original producer is cancelled.
- It allows the downstream to continue, receiving the error event.

The `catch` operator transforms runtime exceptions into predictable events within your stream, creating fault-tolerant pipelines.

The Blueprint Revisited: A System Understood



1. **Ingress**
   `callbackFlow` bridges the "hot" WebSocket.
2. **Validation**
   `map` & `filter` create a declarative processing chain on `Dispatchers.Default`.
3. **Concurrency**
   `flatMapMerge` transforms requests into concurrent, live price streams on `Dispatchers.IO`.
4. **Resilience**
   `catch` ensures that a failure in one price stream doesn't destroy the entire client connection.
5. **Routing**
   The unique Client Request ID, carried through the entire pipeline, ensures the final `PriceResponse` is sent to the correct client.

By composing Flow operators, we've built a sophisticated, non-blocking, and resilient real-time system with clear, readable code.

A Different Primitive: When Should We Use Channels?

A **Channel** provides a way to transfer a stream of values between ***separate coroutines***.

Mental Model

A Channel is conceptually very similar to a `BlockingQueue`, but it uses a **suspending** `send` instead of a blocking `put`, and a **suspending** `receive` instead of a blocking `take`.

Key Characteristics

- **Hot**: A Channel is an active entity. Elements can be sent to it regardless of whether a receiver is ready (if buffered).
- **Communication Primitive**: Its primary purpose is communication and synchronization, not data transformation.
- **Buffering**: Can be unbuffered (rendezvous) or buffered (`Channel(capacity = 4)`), allowing senders to proceed without an immediate receiver.

Basic Usage

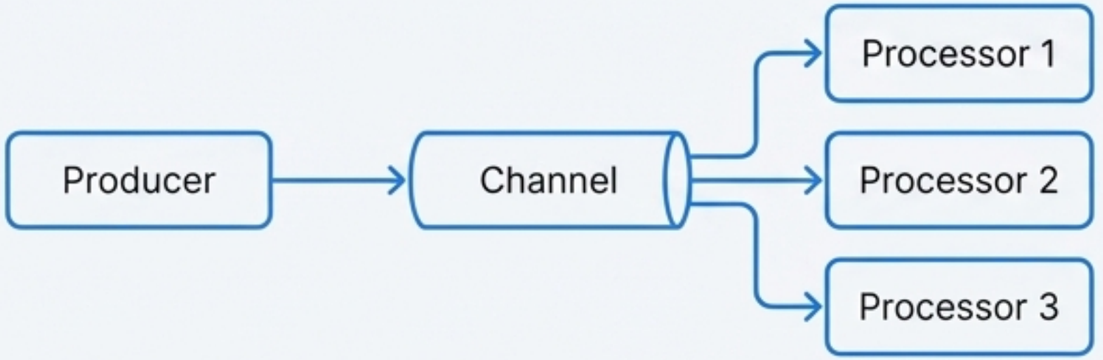
```
val channel = Channel<Int>()

launch { // Coroutine 1 (Producer)
    for (x in 1..5) channel.send(x * x)
    channel.close()
}

launch { // Coroutine 2 (Consumer)
    for (y in channel) println(y)
}
```

Channels are not for data processing pipelines; they are for managing communication between concurrent coroutines.

Channels in Action: Work Distribution Patterns

Fan-Out: Distributing Work

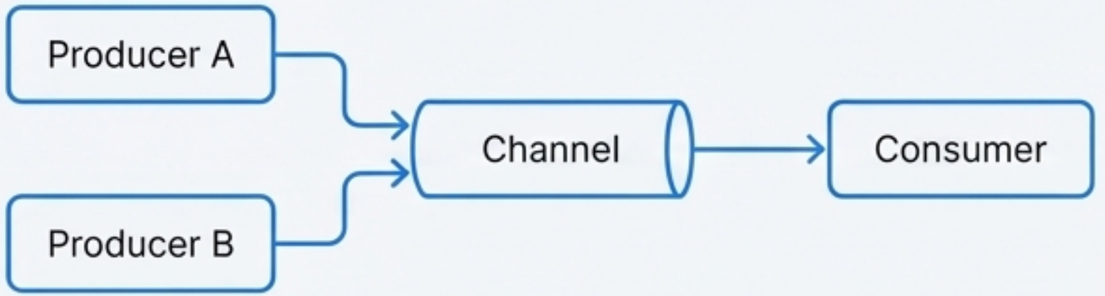
Use Case: One producer sends tasks to a pool of multiple worker coroutines.




```
val producer = produceNumbers() // Produces numbers

repeat(5) { id ->
    launchProcessor(id, producer) // 5 processors consume
}
```

Fan-In: Aggregating Events

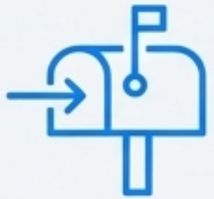
Use Case: Multiple coroutines produce events that are collected and processed by a single consumer.




```
val channel = Channel<String>()
launch { sendString(channel, "foo", 200L) } // Producer 1
launch { sendString(channel, "BAR!", 500L) } // Producer 2

// Single consumer receives from both
repeat(6) { println(channel.receive()) }
```

Key Takeaway: Use fan-out to parallelize work across consumers and fan-in to build event queues that aggregate data from multiple sources.

The Mental Model: Choosing Between Channels and Flows



Channels

Communication Primitive

- **Analogy**: A shared mailbox or queue between workers.
- **Purpose**: Transferring elements between coroutines. Manages the “how” and “when” of communication.
- **Nature**: Always **Hot**.
- **Ownership**: A **Channel** is a concrete, independent entity that can be passed around.
- **Use When**: You need to explicitly manage work distribution (fan-out), event aggregation (fan-in), or state transfer between concurrent actors.

Flows

Data Processing Abstraction

- **Analogy**: A declarative blueprint for a data assembly line.
- **Purpose**: Defining a sequence of asynchronous operations on a stream of data.
- **Nature**: **Cold** by default.
- **Ownership**: A **Flow** is a definition; the data doesn’t “flow” until a terminal operator is called.
- **Use When**: You are building a reactive pipeline to process data from a single source, applying transformations, and handling results.

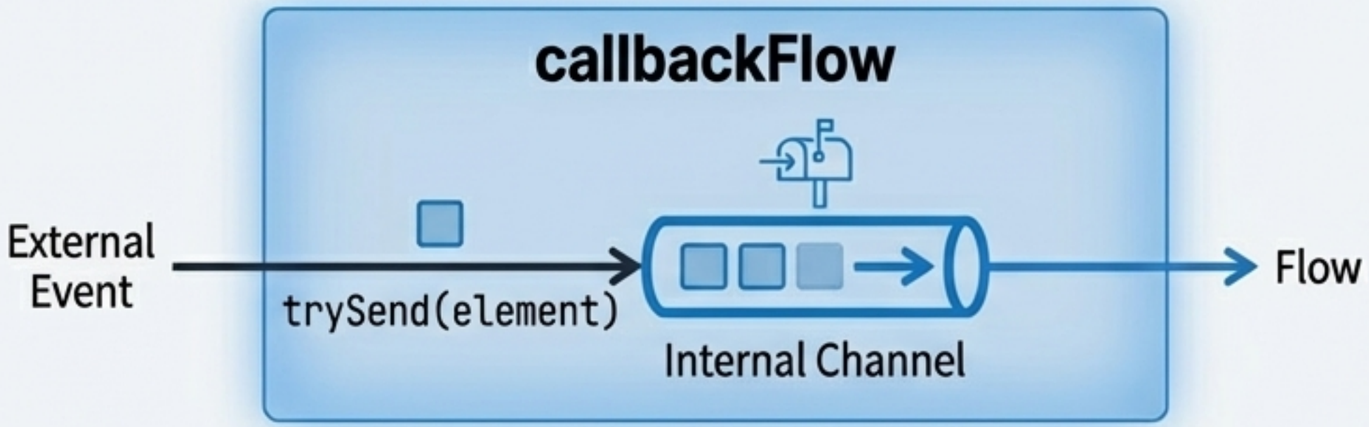
Ask yourself: Am I *communicating between coroutines* (Channel) or *processing a stream of data* (Flow)?

The Hidden Connection: How Hot Flows Are Powered by Channels

High-level abstractions are often built on powerful low-level primitives. Hot flows in Kotlin are a prime example.

The Mechanism

- Builders like `callbackFlow` and operators like `shareIn` (which creates a `SharedFlow`) use a `Channel` internally.
- This internal channel is what provides the **buffering**, multicast behavior, and decoupling of producer from consumer that defines a **hot stream**.
- Example: `callbackFlow`: When you call `trySend`, you are adding an element to its internal channelıel. This channel has a default capacity of 64 elements.



Why This Matters

- It explains the source of **backpressure** in hot flows: if the internal channel's buffer is full, the producer will suspend (with `send`) or drop values (with `trySend`).
- It shows that Flows are a higher-level, safer, and more declarative API for the common use cases that might otherwise be built manually with Channels.

Key Takeaway: Flows provide a powerful and safe abstraction over the lower-level communication mechanics handled by Channels.

Synthesis: The Right Abstraction for the Task

Recap of the Journey

- We started with a complex, real-world problem: a real-time pricing engine.
- We built an elegant and resilient solution by composing declarative **Flow** operators, demonstrating their power for reactive data processing.
- We then explored **Channels** as a distinct tool for explicit inter-coroutine communication and work distribution patterns like fan-in and fan-out.

The Core Principle

The choice is not about which is “better,” but which provides the right level of abstraction for the job at hand.

Start with Flows: For most data processing and reactive programming tasks, the **Flow** API is safer, more declarative, and less error-prone.

Use Channels when needed: When your problem is fundamentally about synchronization, work distribution, or communication between independent, concurrent actors, **Channels** offer the precise control you need.

Mastering both primitives and, more importantly, understanding when to use each, unlocks the full potential of Kotlin Coroutines for building sophisticated, scalable, and modern asynchronous systems.

Choose Flows for data, Channels for communication.